0347. Top K Frequent Elements

| Category: Arrays & Hashing | LeetCode: https://leetcode.com/problems/top-k-f

Problem

- Given nums and k, return the k most frequent elements (answer is unique, any order).
- Example: nums = [1,1,1,2,2,3], k = 2 → [1,2].
- Follow-up: do better than O(n log n).

Intuition

- First count frequencies with a HashMap.
- A frequency is at most n, so we can use bucket sort: bucket[f] holds values that occur f times.
- Walking buckets from high to low yields the top k in linear time.

Brute Force

- Count, then sort the distinct values by frequency: O(n log n).
- A min-heap of size k gives O(n log k), a good alternative to mention.

Optimal Approach

- Count: freq[x]++.
- Create n+1 buckets; put each value into bucket[freq].
- Scan buckets from n down to 1, collecting values until we have k.

Worked Example

```
nums = [1,1,1,2,2,3], k = 2.

freq: 1->3, 2->2, 3->1
bucket[3]=[1] bucket[2]=[2] bucket[1]=[3]
scan f=6..1: f=3 take 1 -> [1]; f=2 take 2 -> [1,2]  k reached
```

Java Solution

```java
public class TopKFrequentElements {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.merge(x, 1, Integer::sum);
        List<List<Integer>> bucket = new ArrayList<>();
        for (int i = 0; i <= nums.length; i++) bucket.add(new ArrayList<>());
        for (var e : freq.entrySet()) bucket.get(e.getValue()).add(e.getKey());
        int[] res = new int[k];
        int idx = 0;
        for (int f = nums.length; f >= 1 && idx < k; f--)
            for (int v : bucket.get(f)) {
                if (idx == k) break;
                res[idx++] = v;
            }
        return res;
    }
}
```

Complexity

- Time: O(n).
- Space: O(n) for the map and buckets.

Pitfalls

- Buckets need size n+1 because a frequency can equal n.
- Stop exactly at k; do not take a whole bucket blindly.